

Attunity

Last updated by | Markham Neil < nMarkham@anglianwater.co.uk > | Thu, 24 Jan 2019 08:59:58 GMT

Databricks Attunity Ingestion Notebook

This document aims to describe the functionality of the Databricks Attunity ingestion notebook and provide guidance for development community on its usage.

Purpose

The sole purpose of this notebook is to provide processing functionality for the data that lands in "Raw". The notebook described here lifts the data landed in "Raw" and processes in a consistent manner into "Base".

Description

The Databricks Attunity Ingestion notebook is designed to be a single notebook that will process all entities listed in BISystem as data source provided by SAP Attunity. Azure Data Factory will acquire a list of entities from BISystem and in-turn, call the notebook for processing into "Base".

Parameters

The only parameter is the entity ID (entity_id). When automated via Azure Data Factory, this will be picked up as described above from BISystem, however, when testing, a developer will be required to manually run the Azure Data Factory Pipeline <ENTER PIPELINE NAME HERE> and manually feed in the required entity ID.

Development Process

When running the notebook for the first time, the notebook will attempt to gather all the require entity's column meta data and automatically insert into BISystem, creating Version 1 of the entity column meta data. One this first run, the notebook will not process data into "Base". The developer must review this entity column meta data, checking it's validity, once confirmed correct within the entity, the developer must set the "developer_validated" column to 1. This will then allow the notebook to process data from "Raw" to "Base" on subsequent executions.

Processing Data in Raw

In order to ensure all files are successfully processed into "Base", some reorganisation of the files contained within "Raw" must occur first. As an example, data will be landed from Attunity into the following folder within the Azure Data Lake Store (ADLS);

- adl://awsawbiattunityneudev01.azuredatalakestore.net/root/Raw/CommercialInConfidence/Sap/Csv/**Unprocessed**/SAP.QMEL/Load0000001.csv

Before processing into "Base", this and all other files in this directory are moved into the following location;

- adl://awsawbiattunityneudev01.azuredatalakestore.net/root/Raw/CommercialInConfidence/Sap/Csv/**Processing**/SAP.QMEL/Load0000001.csv

Once the data has been processed into "Base", the file is then moved from the above location into the following;

- `adl://awsawbiattunityneudev01.azuredatalakestore.net/root/Raw/CommercialInConfidence/Sap/Csv/Processed/SAP.QMEL/Load0000001.csv`

Initial and Change Tracking (ct)

With Attunity an initial snapshot is taken of how the data looks at the point in time when first activated within the Attunity program, this always lands into a ADLS folder that corresponds to the above example. However, subsequent transactional changes are tracking into a separate folder, illustrated below;

- `adl://awsawbiattunityneudev01.azuredatalakestore.net/root/Raw/CommercialInConfidence/Sap/Csv/Unprocessed/SAP.QMEL__ct/Load0000001.csv`

These files are processed in the same way, moving across an "Unprocessed" and "Processed" collection of files. The only difference between these two sets of data is that Attunity will inject header meta data into the change tracking files.

When the Initial and Change Tracking files are processed for any given entity, they are expected to have the same structure with the only exception description previously. In order for both the Initial and Change Tracking to exactly align, we inject the same meta data columns into any initial file found so that the data sets can be merged successfully. The main advantage of this approach is that no separate process is required for Initial or Change Tracking. If however the structures genuinely do not align, a developer will need to process any files that match the existing version 1 schema (by moving the ones that do not match out into a separate folder (not picked up by Databricks processing) and reprocess.

Changing Entity Column Versions

If the above scenario occurs described towards the end of the "Initial and Change Tracking" section then a new version of the entity columns will need to be created. Simple setting the `entity_column_version` column within the entity table to 2 will now try and detect version 2 columns in entity columns table. By design, these will not be found, and the process described above where these are automatically populated and then validation must be repeated.

Data Cleansing

The main value add of the "Base" layer within ADLS is data cleansing, data should be consistently cleansed and ready for data processing. The data cleansing actions that occur within the notebook are described below;

- Trimming spaces from left and the right of text
- Replacing a single space column (common SAP occurrence) with a '?'
- Replacing null with '?'
- Removing duplicate rows within the processing batch